
UniSaS – Guia de Estudo e Apresentação do Projeto

1. TECNOLOGIAS UTILIZADAS

Backend

Node.js

Node.js é o ambiente que executa JavaScript fora do navegador, ou seja, no servidor. No projeto UniSAS, todo o backend é executado utilizando Node.js.

Onde verificar:

`backend/src/server.js`

Função principal: Inicializar e manter o servidor da aplicação em execução.

Express

Express é um framework para Node.js que simplifica a criação de APIs REST, gerenciamento de rotas, middlewares e requisições HTTP.

Onde verificar:

`backend/src/app.js`

Função principal:

- Definição de rotas
- Configuração de middlewares
- Tratamento de requisições HTTP
- Integração entre frontend e backend

Supabase (PostgreSQL)

Supabase é uma plataforma baseada em PostgreSQL utilizada como banco de dados principal do sistema.

No UniSAS, ele substitui um ORM tradicional, permitindo consultas diretas através de sua SDK JavaScript.

Onde verificar:

`backend/src/config/database.js`

Função principal:

- Armazenamento dos dados do sistema
 - Consultas SQL
 - Relacionamentos entre tabelas
 - Persistência das informações
-

JWT (JSON Web Token)

Biblioteca utilizada para autenticação baseada em tokens.

Onde verificar:

- Geração:

`backend/src/services/authService.js`

- Validação:

`backend/src/middlewares/authMiddleware.js`

Função principal:

- Identificar usuários autenticados
 - Evitar consultas ao banco a cada requisição
 - Garantir segurança nas rotas protegidas
-

bcrypt

Biblioteca utilizada para criptografia de senhas.

Onde verificar:

`backend/src/services/authService.js`

Função principal:

- Transformar senhas em hashes
 - Impedir armazenamento de senhas em texto puro
 - Comparar senha digitada com senha armazenada
-

Redis

Banco de dados em memória utilizado como cache.

Onde verificar:

- `backend/src/config/cache.js`
- `backend/src/middlewares/cacheMiddleware.js`

Função principal:

- Armazenar respostas temporariamente
- Reduzir consultas ao banco principal
- Melhorar desempenho da aplicação

Multer

Middleware utilizado para upload de arquivos.

Onde verificar:

`backend/src/middlewares/uploadMiddleware.js`

Função principal:

- Receber arquivos enviados pelo navegador
 - Manipular PDFs
 - Encaminhar arquivos ao Supabase Storage
-

`express-rate-limit`

Biblioteca de proteção contra ataques de força bruta.

Onde verificar:

`backend/src/middlewares/rateLimitMiddleware.js`

Função principal:

- Limitar requisições por IP
 - Proteger rotas críticas
 - Evitar abuso da API
-

Frontend

React

Biblioteca JavaScript para construção de interfaces.

Onde verificar:

- `frontend/src/pages/`
- `frontend/src/components/`

Função principal:

- Construção da interface gráfica
 - Componentização
 - Atualização dinâmica da tela
-

React Router DOM

Biblioteca responsável pela navegação entre páginas.

Onde verificar:

`frontend/src/App.jsx`

Função principal:

- Gerenciar rotas
- Criar SPA (Single Page Application)
- Navegação sem recarregar a página

Vite

Ferramenta moderna para desenvolvimento
React.

Onde verificar:

- `frontend/vite.config.js`
- `frontend/package.json`

Função principal:

- Build da aplicação
- Ambiente de desenvolvimento rápido
- Otimização para produção

Docker e Docker Compose

Ferramentas para containerização da
aplicação.

Onde verificar:

- `backend/Dockerfile`
- `frontend/Dockerfile`

- `docker-compose.yml`

Função principal:

- Padronizar ambiente
 - Facilitar deploy
 - Executar múltiplos serviços simultaneamente
-

2. ARQUITETURA DO SISTEMA

Arquitetura Utilizada

O UniSAS utiliza uma **Arquitetura Monolítica em Camadas**, mantendo frontend e backend separados fisicamente.

Fluxo Geral

Frontend (React + Vite)



HTTP REST (JSON + JWT)



Backend (Node.js + Express)



Supabase (PostgreSQL) + Redis (Cache)

Camadas do Backend

1. Routes

Localização:

`backend/src/routes`

Responsável por:

- Definir URLs da API
- Definir middlewares
- Direcionar requisições aos controllers

Analogia: Recepcionista de uma empresa.

2. Controllers

Localização:

`backend/src/controllers`

Responsável por:

- Receber requisições

- Extrair dados
- Chamar serviços
- Retornar respostas

Analogia: Gerente responsável por coordenar o atendimento.

3. Services

Localização:

`backend/src/services`

Responsável por:

- Regras de negócio
- Consultas ao banco
- Processamento dos dados

Analogia: Especialista que realiza o trabalho.

4. Middlewares

Localização:

`backend/src/middlewares`

Responsável por:

- Autenticação
- Cache
- Validação
- Logs
- Tratamento intermediário

Analogia: Segurança que verifica se alguém pode entrar.

Estrutura do Frontend

pages/

Contém telas completas.

Exemplos:

- DashboardAluno
 - DashboardProfessor
 - GerenciarTurmasPage
-

components/

Componentes reutilizáveis.

Exemplos:

- TopBar
 - StatCard
 - TabNav
-

`services/`

Comunicação com a API.

Exemplos:

- `api.js`
 - `notaService.js`
-

`utils/`

Funções auxiliares.

Exemplos:

- Formatação de dados
 - Helpers
 - Conversões
-

3. PADRÕES DE PROJETO

Singleton

Conceito

Garante uma única instância compartilhada em toda a aplicação.

Onde é utilizado

- `database.js`
- `cache.js`

Benefício

Evita múltiplas conexões com banco e cache.

Service Layer

Conceito

Separa regras de negócio dos controllers.

Onde é utilizado

Toda a pasta:

`backend/src/services`

Benefício

Facilita manutenção e reutilização.

Middleware Pattern (Chain of Responsibility)

Conceito

A requisição percorre uma cadeia de responsabilidades.

Benefício

Permite separar:

- Autenticação
 - Validação
 - Cache
 - Segurança
-

Factory Function

Conceito

Função que cria outras funções.

Onde é utilizado

`cacheMiddleware.js`

Benefício

Evita repetição de código.

MVC (Model-View-Controller)

Estrutura no UniSAS

MV UniSAS
C

Mod Services
el +
 Supabas
 e

View React

Cont Controller
rolle s
r Express

Benefício

Separação clara de responsabilidades.

Repository-Like

Conceito

Centralização do acesso aos dados.

Onde é utilizado

Cada arquivo dentro de:

`backend/src/services`

Benefício

Facilita futuras mudanças de banco de dados.

4. PROGRAMAÇÃO ORIENTADA A ASPECTOS (POA)

O que é POA?

POA (Programação Orientada a Aspectos) é uma abordagem que separa preocupações transversais da lógica principal do sistema.

Exemplos:

- Logging
- Autenticação

- Cache
 - Tratamento de erros
 - Validação
-

Aspectos Implementados no UniSAS

1. Logging

Responsável por registrar:

- Data
 - Hora
 - Método HTTP
 - Rota
 - Tempo de resposta
-

2. Autenticação

Validação de JWT para rotas protegidas.

3. Tratamento de Erros

Centralização do tratamento de exceções.

4. Rate Limiting

Controle de requisições por usuário/IP.

5. Cache

Armazenamento temporário de respostas utilizando Redis.

6. Validação

Verificação de dados antes do processamento.

Como explicar POA ao professor

"O sistema implementa Programação Orientada a Aspectos através de middlewares. Aspectos transversais como autenticação, cache, logging, tratamento de erros, validação e rate limiting foram separados da lógica de negócio e aplicados de forma reutilizável nas rotas."

5. BANCO DE DADOS

Estrutura Geral

O UniSAS utiliza três mecanismos de persistência:

Tecnologia	Tipo	Finalidade
Supabase PostgreSQL	Relacional	Dados do sistema
Redis	Chave-Valor	Cache
Supabase Storage	Arquivos	PDFs e documentos

PostgreSQL via Supabase

Funções principais

- Armazenar usuários
- Armazenar turmas
- Armazenar conteúdos
- Armazenar atividades
- Armazenar entregas

Principais tabelas

usuarios

- id
- nome
- email
- senha_hash
- perfil
- turma_id
- matricula
- cpf

turmas

- id
- nome
- turno
- ano_letivo

atividades

- id
- titulo
- descricao
- prazo
- turma_id
- arquivo_url

entregas

- id
- aluno_id
- atividade_id
- nota
- data_entrega

conteudos

- id
- professor_id
- turma_id
- titulo

- descricao
 - arquivo_url
-

Redis

Objetivo

Melhorar o desempenho da aplicação através de cache.

Funcionamento

1. Usuário solicita dados.
 2. Redis verifica se já possui resposta.
 3. Se existir, retorna imediatamente.
 4. Caso contrário:
 - consulta o Supabase;
 - salva resultado no Redis;
 - retorna ao usuário.
-

Supabase Storage

Objetivo

Armazenar arquivos PDF enviados por professores e alunos.

Fluxo

1. Usuário envia PDF.
 2. Multer recebe arquivo.
 3. Supabase Storage salva o arquivo.
 4. URL pública é gerada.
 5. URL é armazenada no banco.
-

Variáveis de Ambiente

Arquivo:

`backend/.env`

Principais variáveis:

- PORT
 - SUPABASE_URL
 - SUPABASE_SERVICE_KEY
 - JWT_SECRET
 - REDIS_URL
 - FRONTEND_URL
-

CONCLUSÃO

O UniSAS foi desenvolvido utilizando uma arquitetura moderna baseada em Node.js, Express, React, PostgreSQL (Supabase) e Redis. O projeto aplica conceitos importantes de Engenharia de Software, como Arquitetura em Camadas, Design Patterns, Programação Orientada a Aspectos (POA), autenticação JWT, cache com Redis e armazenamento de arquivos com Supabase Storage, garantindo organização, escalabilidade, segurança e facilidade de manutenção.

Conclusão Geral do Projeto UniSAS

O UniSAS foi desenvolvido seguindo conceitos modernos de desenvolvimento de software, utilizando uma arquitetura organizada em camadas, tecnologias amplamente utilizadas pelo mercado e boas práticas de engenharia de software. A estrutura adotada proporciona maior

manutenção, escalabilidade, segurança e reutilização de código.

Backend

O backend foi desenvolvido utilizando **Node.js** e **Express**, responsáveis pelo processamento das regras de negócio e comunicação com o banco de dados. A aplicação foi estruturada em camadas de **Routes, Controllers, Services e Middlewares**, permitindo uma clara separação de responsabilidades. Além disso, foram utilizadas bibliotecas como **JWT** para autenticação, **bcrypt** para criptografia de senhas, **Multer** para upload de arquivos e **express-rate-limit** para proteção contra ataques de força bruta.

Frontend

O frontend foi construído com **React**, utilizando uma arquitetura baseada em componentes reutilizáveis e páginas organizadas. A navegação entre telas é

realizada através do **React Router DOM**, proporcionando uma experiência de aplicação de página única (SPA). O projeto também utiliza **Vite**, ferramenta responsável pela compilação e otimização da aplicação, garantindo maior desempenho durante o desenvolvimento e na versão final.

Banco de Dados

O sistema utiliza o **Supabase (PostgreSQL)** como banco de dados principal, armazenando todas as informações relacionadas a usuários, turmas, conteúdos, atividades e entregas. Para otimizar o desempenho das consultas, foi implementado o **Redis**, atuando como mecanismo de cache temporário. Além disso, o **Supabase Storage** é utilizado para armazenamento de arquivos PDF, permitindo o gerenciamento de documentos dentro da plataforma.

Arquitetura do Sistema

A arquitetura adotada é uma **Arquitetura Monolítica em Camadas**, onde frontend e backend permanecem separados fisicamente, mas integrados através de uma API REST. Essa abordagem facilita a organização do sistema, permitindo que cada camada tenha uma responsabilidade específica e reduzindo o acoplamento entre os componentes.

Padrões de Projeto

Durante o desenvolvimento foram aplicados diversos padrões de projeto para aumentar a qualidade e a manutenção do código.

Entre eles destacam-se:

- **Singleton**, para garantir uma única instância das conexões com banco de dados e cache;
- **Service Layer**, separando as regras de negócio dos controllers;
- **Middleware Pattern (Chain of Responsibility)**, organizando

validações, autenticação e demais processamentos intermediários;

- **Factory Function**, permitindo a criação dinâmica de comportamentos reutilizáveis;
- **MVC Adaptado**, separando controle, dados e interface;
- **Repository-like**, centralizando o acesso aos dados por entidade.

Esses padrões tornam o sistema mais organizado, reutilizável e preparado para futuras evoluções.

Programação Orientada a Aspectos (POA)

O projeto também implementa conceitos de **Programação Orientada a Aspectos (POA)** através da utilização de middlewares. Aspectos transversais como **autenticação, logging, cache, validação de dados, tratamento de erros e controle de requisições** foram isolados da lógica principal da aplicação, evitando repetição

de código e aumentando a modularidade do sistema. Dessa forma, as regras de negócio permanecem mais limpas e fáceis de manter.

Considerações Finais

Em conjunto, as tecnologias, padrões arquiteturais e boas práticas adotadas no UniSAS demonstram uma preocupação com qualidade, segurança, desempenho e escalabilidade. O projeto apresenta uma estrutura profissional, alinhada aos conceitos estudados na área de Desenvolvimento de Software, evidenciando a aplicação prática de conhecimentos de backend, frontend, banco de dados, arquitetura de software, padrões de projeto e Programação Orientada a Aspectos.